



Roteiro de Live Coding: Teste de Integração no VeterinarioService



Introdução Conceitual (5 minutos)

Até agora, nós testamos o VeterinarioService de forma **isolada**. Usamos o Mockito para criar um "dublê" do banco de dados. Mas no mundo real, precisamos ter certeza de que o nosso Serviço consegue conversar perfeitamente com o Banco de Dados verdadeiro.

É aqui que entram os **Testes de Integração**!

Neste tutorial, vamos remover o Mockito e plugar o nosso Serviço diretamente no banco de dados H2.



O que muda do Teste de Unidade (Mock) para o de Integração?

Característica	Teste de Unidade (com Mockito)	Teste de Integração (Sem Mock)
Anotação da Classe	@ExtendWith(MockitoExtension.class)	@SpringBootTest
Injeção do Serviço	@InjectMocks	@Autowired
Injeção do Repositório	@Mock	Não é obrigatório (mas se usar, é @Autowired)
Preparação (Arrange)	Nós "ensinamos" o Mock com when(...)	Usamos os dados reais do import.sql
Verificação (Assert)	Usamos assertEquals e verify(...)	Usamos apenas assertEquals, pois o dado realmente vai pro banco!
Velocidade	Frações de segundo	Leva alguns segundos (sobe o Spring inteiro)

Passo a Passo da Transformação

Passo 1: Criando a Classe e as Anotações de Contexto

Vamos criar uma nova classe chamada `VeterinarioServiceITest.java` (o "IT" vem de *Integration Test*) no pacote de testes.

Para que o Spring Boot levante todo o seu contexto (o banco H2, o Hibernate, as injeções de dependência), precisamos trocar a anotação principal da classe.

Além disso, adicionamos a anotação **@Transactional**. Ela é uma "mágica" nos testes de integração: garante que toda vez que um teste salvar ou apagar algo, o Spring faça um *Rollback* (desfaça a ação) no final. Assim, um teste não estraga o banco de dados para o próximo teste!

// ANTES (Unitário): @ExtendWith(MockitoExtension.class)

@SpringBootTest // 1. Sobe o Spring inteiro, o banco H2 e o Hibernate

@Transactional // 2. Desfaz (Rollback) qualquer alteração no banco ao fim de cada teste

```
public class VeterinarioServiceIT {
```

```
    @Autowired
```

```
    private VeterinarioService service; // Injeção real da classe que vamos testar
```

```
    @Autowired
```

```
    private VeterinarioRepository repository; // Injeção real para nos ajudar a conferir o banco
```

```
}
```

👉 **Dica:** A importância do `@Transactional`. Sem ele, um teste que salva um veterinário "suja" o banco para o próximo teste, quebrando a independência deles.

Passo 2: Testando a Busca (Aproveitando o `import.sql`)

Nós não precisamos mais do `when()`, pois o banco de dados vai responder de verdade. Nós vamos usar os dados que já existem no seu arquivo `import.sql` (Vamos supor que o Conceição já esteja salvo lá com o ID 1).

```
@Test
```

```
void deveBuscarVeterinarioPorIdComSucessoComLimiteCaracteres() {
```

```
    // Arrange
```

```
    Integer idExistente = 1;
```

```
    // Act - Chamada real que vai bater lá na tabela do H2
```

```
    Optional<Veterinario> resultado = service.buscaVeterinariosPeloId(idExistente);
```

```
    // Assert
```

```
    assertTrue(resultado.isPresent());
```

```
    // verifica a regra de negocio de retornar apenas os 10 primeiros caracteres.
```

```
    assertEquals("Conceição", resultado.get().getNome());
```

```
    // Pergunta: "Cadê o verify()?"
```

```
    // Resposta: Não existe verify! Agora validamos o dado real retornado.  
}
```

Passo 3: Testando a Inserção (Cenário de Escrita)

Agora, vamos testar o método `salvar`. É aqui que o banco de dados funcionando de verdade, gerando IDs incrementais.

```
@Test  
void deveSalvarVeterinarioNoBancoDeDados() {  
    // Arrange - Objeto novo, sem ID (nulo)  
    Veterinario novoVet = new Veterinario(null, "Dra. Marcia", "marcia@gmail.com", "Grandes  
    Animais", BigDecimal.valueOf(5500.0));  
  
    // Act  
    Veterinario salvo = service.salvar(novoVet);  
  
    // Assert  
    assertNotNull(salvo.getId(), "O banco H2 deveria ter gerado um ID automático!");  
    assertEquals("Dra. Marcia", salvo.getNome());  
  
    // Prova Real: Usando o repository para checar se ele está gravado de verdade  
    Optional<Veterinario> vetNoBanco = repository.findById(salvo.getId());  
    assertTrue(vetNoBanco.isPresent());  
    assertEquals("marcia@gmail.com", vetNoBanco.get().getEmail());  
}
```

Passo 4: Testando Regra de Negócio e Exceção (O fluxo alternativo)

Lembra do desafio de não apagar um veterinário inexistente? Vamos ver como ele se comporta na integração. Se o serviço tentar buscar um ID que não está no H2, ele deve lançar a exceção.

```
@Test  
void deveLancarExcecaoAoApagarIdNaoExistente() {  
    // Arrange  
    Integer idInexistente = 9999; // ID que não mapeamos no import.sql  
    int quantidadeOriginal = 2;  
    // Act & Assert  
    RuntimeException e = assertThrows(RuntimeException.class, () -> {  
        service.apagarPorId(idInexistente);  
    });  
    int quantidadeAtual = service.buscaTodosVeterinarios().size();  
    assertEquals("Veterinário com ID 9999 não foi encontrado no banco de dados. Operação  
    de exclusão cancelada.", e.getMessage());  
    // Pergunta: "Como garantir que o banco não apagou nada errado?"  
    // Podemos checar se a contagem de registros continua igual!  
    assertEquals(quantidadeOriginal, quantidadeAtual);  
}
```



3 Observações:

1. **O "Peso" do Inicializador:** Quando você clicar em *Run* no teste pela primeira vez, o Spring é carregado, as tabelas são criadas pelo Hibernate e gasta mais tempo (geralmente de 2 a 5 segundos). O teste de Mockito rodava instantaneamente.
2. **A ausência do Mockito:** O código do teste ficou muito mais limpo visualmente porque não há configurações de comportamento (*when*) nem espionagem de métodos (*verify*).